adapter.py

`src/moe_pruning/models/switch_transformers/adapter.py · 371 lines · module moe_pruning.models.switch_transformers.adapter`

What it does

Everything specific to Hugging Face Switch Transformers: how sparse layers are found, how routing works and how expert inputs and outputs are observed.

How it works

- Finds sparse layers structurally (a module with a `router` and an `experts` container), in encoder and decoder; layer ids look like `encoder.block.1`.
- Routing is top-1 with a per-sequence expert capacity; dropped tokens get expert index -1 and gate 0.
- The gate is applied by the parent (`SwitchTransformersExperts.forward` multiplies each expert output once), so a replacement expert returns its un-gated output only.
- Instrumentation hooks the feed-forward block, the MoE module, the router and each expert to build `MoEForwardRecord`s; expert outputs are mapped back to their tokens.
- The reroute mask is a router forward hook that repeats HF's top-1 + capacity assignment with removed experts excluded; tokens keep their original gates if their expert is kept.
- `describe()` produces the architecture report written by `inspect_model.py`.

Defined in this file

Extracted from the source code (classes, public functions and methods).

Name	Description
`class SwitchTransformersAdapter`	MoEAdapter for Hugging Face Switch Transformers.
`.discover_layers(model)`	Find all sparse MoE layers by inspecting the loaded model.
`.get_moe_module(model, layer)`	The sparse MoE module of a layer.
`.get_router(model, layer)`	The router module of a layer.
`.get_expert(model, layer, expert_id)`	Module computing one expert (original, view or replacement).
`.set_expert(model, layer, expert_id, module)`	Install `module` in the expert slot and return the previous module.
`.build_original_expert(model, layer)`	New (uninitialised) expert module of the model's original expert class.
`.expert_state_prefix(layer, expert_id)`	State-dict key prefix of an expert slot (ends with a dot).
`.route(model, layer, moe_input, apply_mask)`	Return (expert_index [B,S,K], gate [B,S,K]) exactly as used by the layer.
`.set_routing_mask(model, layer, expert_ids)`	Make `expert_ids` unreachable: their tokens go to the best remaining expert.
`.get_routing_mask(model, layer)`	Experts currently made unreachable by the reroute mask.
`.run_moe_layer(model, layer, moe_input)`	Execute only the sparse MoE module on `moe_input` ([B,S,D]).
`.instrument(model, layers, callback)`	Call `callback` with a `MoEForwardRecord` per layer forward.
`.describe(model)`	Machine-readable architecture report.

Connections

Uses: `moe_pruning.model.architecture`

Used by: `src/moe_pruning/models/switch_transformers/__init__.py`